



EcoQuest

Plataforma de concienciación ambiental gamificada

👥 **Equipo Prompt** · IES Doctor Balmis · DAM 2025–2026

📱 App Android · ☁ API REST · 💻 Panel de Administración WPF



Introducción

¿Por qué EcoQuest?

- 🌍 La gente quiere actuar, pero no sabe cómo
- 🏠 Falta de canales para organizarse localmente
- 🎮 Las apps verdes existentes no enganchan

Nuestra propuesta

- Comunidades vecinales conectadas
- Eventos y retos ecológicos reales
- Sistema de puntos que motiva la participación



Objetivo

Fomentar hábitos sostenibles mediante comunidades locales, eventos reales y recompensas.



Usuarios

Ciudadano usa la app móvil

Administrador gestiona desde el panel WPF



Conectado

Móvil y escritorio comparten datos en tiempo real a través de la API REST.



Arquitectura del sistema

App Android

Kotlin · Jetpack Compose · Hilt · Room · Retrofit



REST / JWT



API REST

Spring Boot 3.4 · Java 21 · Puerto 9000



JPA / Hibernate



Base de datos

PostgreSQL (prod) · H2 (dev)



REST / JWT



Panel Admin WPF

C# · .NET · MVVM

Android — patrón MVVM

Compose UI



eventos / estado

ViewModel ← Hilt DI



Repository



Room DB

(offline)



Retrofit

(API REST)

- Cada pantalla: `Screen` + `ViewModel` + `UiState` + `Event`
- Estado reactivo con `Flow` y `mutableStateOf`
- Navegación type-safe con `@Serializable`

Funcionalidades — App Android



Comunidades

- Explorar y unirse a comunidades locales
- Mural compartido por comunidad
- El administrador aprueba comunidades nuevas

Eventos comunitarios

- Feed filtrable en tiempo real
- Tipos: Comunitario · Noticia · Urgente
- Cualquier miembro puede crear eventos



Funcionalidades — App Android



Tienda de Puntos

- Los usuarios acumulan puntos participando
- Catálogo de recompensas canjeables
- Vista de puntos disponibles en tiempo real

Perfil de usuario

- Datos personales editables
- Historial de actividad
- Foto de perfil

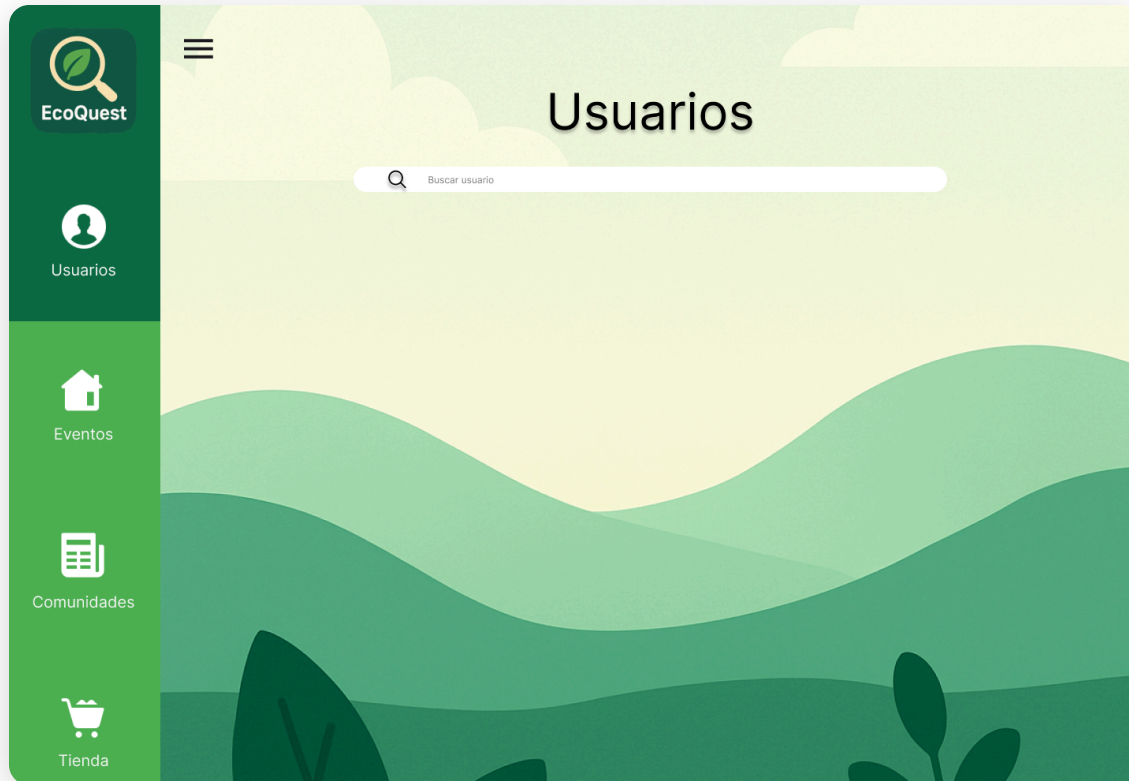


App de Escritorio

Panel de administración — WPF · C# · MVVM



Panel de Administración



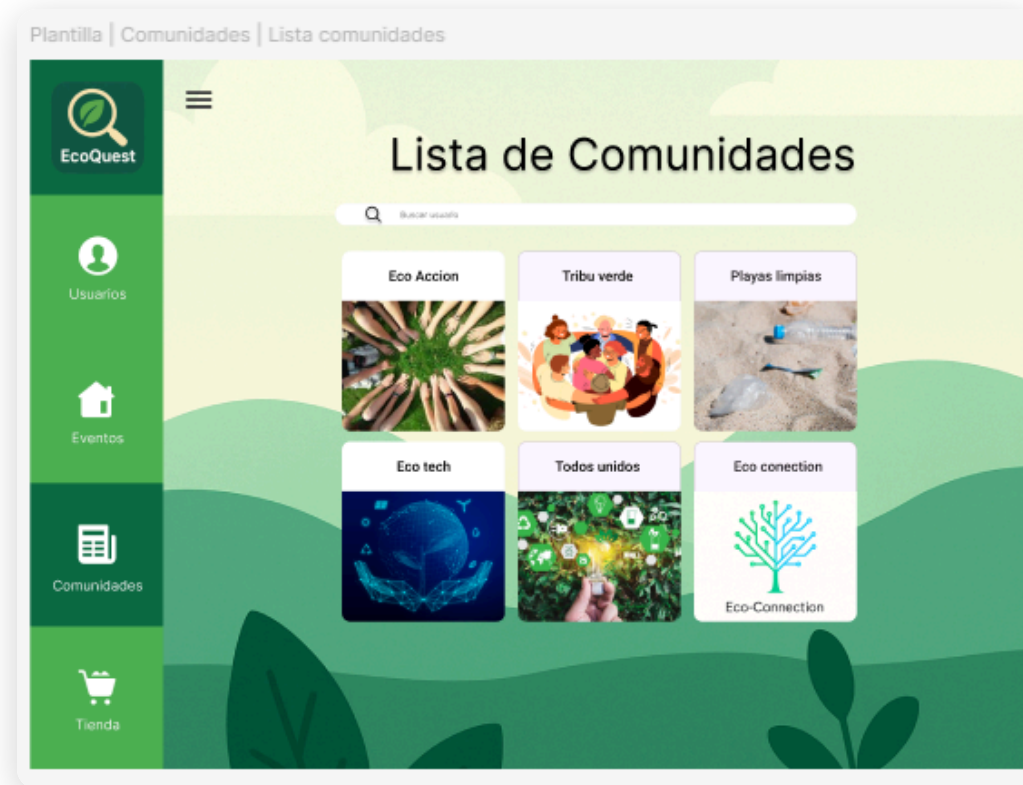
Destinada al administrador

- Gestión completa de usuarios
- Aprobar o rechazar comunidades
- Moderar eventos publicados
- Gestionar complementos de la tienda

 Lo que hace el admin se refleja en tiempo real en la app móvil — comparten la misma API REST y base de datos.



Panel — Gestión de comunidades



Lista de comunidades · Flujo de aprobación de comunidades pendientes



Características técnicas destacables

Jetpack Compose + MVVM

```
@HiltViewModel
class EventosViewModel @Inject constructor(
    private val repo: EventoRepository
) : ViewModel() {

    var state by mutableStateOf(EventosUiState())
    private set

    init {
        viewModelScope.launch {
            repo.getAll().collect { lista ->
                state = state.copy(eventos = lista)
            }
        }
    }
}
```

Estado reactivo: cuando cambia la BD, la UI se actualiza sola.

Autenticación JWT

```
// Retrofit interceptor añade el token
// automáticamente en cada petición
class AuthInterceptor(private val token: String)
    : Interceptor {
    override fun intercept(chain: Chain) =
        chain.proceed(
            chain.request().newBuilder()
                .header("Authorization", "Bearer $token")
                .build()
        )
}
```

Hilt — Inyección de dependencias

@HiltViewModel

@Singleton

@Inject constructor

Sin instanciar manualmente ningún repositorio ni DAO.



Características técnicas destacables

Room — persistencia local

```
@Dao
interface EventoDao {
    @Query("SELECT * FROM eventos")
    fun getAll(): Flow<List<EventoEntity>>

    // IGNORE = idempotente: no sobrescribe
    // datos existentes en reinicios
    @Insert(onConflict = OnConflictStrategy.IGNORE)
    suspend fun insertAllIfAbsent(
        eventos: List<EventoEntity>
    )
}
```

La app funciona **offline** gracias a Room. Retrofit sincroniza cuando hay conexión.

Stack completo

Kotlin Jetpack Compose Hilt Room Retrofit Coil

App Android

Spring Boot 3.4 Java 21 JWT JPA PostgreSQL

API REST

C# WPF .NET MVVM

Panel administrador



Conclusión

Lo que hemos entregado

-  App Android funcional (6 pantallas)
-  API REST con auth JWT
-  Panel de administración WPF
-  Las 3 plataformas conectadas

Posibles ampliaciones

-  Geolocalización de eventos
-  Retos personalizados con IA
-  Notificaciones push
-  Versión web responsive

El equipo — Equipo Prompt

✓ Pros

Reparto claro de tareas por plataforma. Buen uso de Git para integrar el trabajo.

⚡ Retos

Coordinar 3 plataformas distintas con una sola API compartida requirió definir bien los contratos desde el principio.

Aprendizaje principal

Jetpack Compose y MVVM en un proyecto real de tamaño considerable.



¡Gracias!

Esta ha sido nuestra propuesta — EcoQuest

👥 **Equipo Prompt** · IES Doctor Balmis · DAM 2025–2026

🔗 github.com/Prompt-pi2damiesbalmis/PROYECTO12